*Advanced Level: Data Analysis Bootcamp Using Open-Access Program R*

# Session 5: Systematic and literature review

Patrick Njage

2024-04-12

Contents

**PART I: A RAPID, OBJECTIVE, REPRODUCIBLE METHOD FOR PLANNING A SYSTEMATIC SEARCH OF THE SCIENTIFIC LITERATURE ON A GIVEN TOPIC** ..... **1**

**INTRODUCTION** ..... **1**

**SETUP** ..... **1**

**INSTALLATION** ..... **1**

**JOURNAL ARTICLE SEARCH AND RETRIEVAL AND BIBLIOGRAPHY EXPORT** ..... **2**

EXAMPLE ..... 2

**LOADING AND INSPECTION OF RESULTS** ..... **2**

KEYWORDS ..... 4

TITLES AND ABSTRACTS ..... 8

**NETWORK ANALYSIS** ..... **11**

GGRAPH ..... 13

PRUNING ..... 14

*Cumulatively* ..... *16*

*Changepoints* ..... *17*

**GROUPING** ..... **19**

**WRITING A NEW SEARCH** ..... **20**

**CHECKING THE NEW SEARCH** ..... **21**

AGAINST THE NAIVE SEARCH ..... 21

AGAINST GOLD STANDARD RESULTS ..... 22

**PART II: FACILITATING SYSTEMATIC REVIEWS, DATA EXTRACTION AND META-ANALYSIS** ..... **22**

**INTRODUCTION** ..... **22**

**INSTALLATION AND DEPENDENCIES** ..... **23**

**DELEGATING REFERENCE SCREENING EFFORTS TO A TEAM** ..... **23**

SCREENING ABSTRACTS OF REFERENCES ..... 27

MERGING SCREENED REFERENCES ..... 28

**PART III GENERATING PRISMA PLOTS** ..... **29**

GENERATING DIFFERENT PRISMA PLOT LAYOUTS ..... 30

NOTES ON PRISMA PLOTTING SINCE METAGEAR V. 0.1 AND 0.2 ..... 33

**REFERENCES** ..... **33**

## PART I: A rapid, objective, reproducible method for planning a systematic search of the scientific literature on a given topic

### Introduction

Systematic review, meta-analysis and other forms of evidence synthesis are critical to strengthen the evidence base in scholarly questions. Synthesis lags behind the pace of scientific publishing, however, due to time and resource costs which partial automation of evidence synthesis tasks could reduce. Additionally, current methods of retrieving evidence for synthesis are susceptible to bias towards studies with which researchers are familiar. In fields that lack standardized terminology encoded in an ontology research teams can unintentionally exclude articles from the review by omitting synonymous phrases in their search terms. This topic will introduce a quick, objective, reproducible method for generating search strategies that uses text mining and keyword co-occurrence networks to identify the most important terms for a review. The method reduces bias in search strategy development because it does not rely on a predetermined set of articles and can improve search recall by identifying synonymous terms that research teams might otherwise omit.

The R package `litsearchr` provides various functions to help with planning a systematic search of the scientific literature on a given topic. The `litsearchr` package for R is designed to partially automate search term selection from a large number of studies and writing search strategies for systematic reviews. This session demonstrates its utility by demonstrating how the package: (1) Identifies potential keywords through the naive search input, (2) Builds a keyword co-occurrence network to assist with building a more precise search strategy, (3) Uses a cutoff function to identify important changes in keyword importance, (4) Assists with grouping terms into concepts, and (5) Writes a Boolean search as a result of completion of the four previous steps.

## Setup

As well as litsearchr itself, we will use a few other R packages in the tutorial. Let's load them.

```
library(dplyr)
library(ggplot2)
library(ggraph)
library(igraph)
library(readr)
```

## Installation

To use the `litsearchr` package, we need to install it first. `litsearchr` is not (yet) stored in the package repository of CRAN, the Comprehensive R Archive Network. This means that the usual `install.packages()` function or the Install button in RStudio won't find it. Instead, we can install the package from Eliza's (`litsearchr` was created by Eliza Grames) GitHub repository.

The `devtools` package provides a function for installing R packages from GitHub instead of from CRAN. So we can use this.

```
library(devtools)
# to install, uncomment the code below by removing the #
# install_github("elizagrames/litsearchr", ref="main")
```

Now that we have installed `litsearchr` we can load it.

```
library(litsearchr)
```

`litsearchr` is a new package and is currently in development. So we should keep track of which version we are using, in case we later work with a newer version and find that the examples in this tutorial no longer work.

```
packageVersion("litsearchr")

## [1] '1.0.0'
```

## Journal article search and retrieval and bibliography export

### Example

Let's imagine that we want to retrieve journal articles about treating phobias with a combination of medication and cognitive-behavioral therapy (CBT). Our overall topic consists of the three sub-topics medication, CBT and phobia, and we want to get articles that are about all three.

The starting point in `litsearchr` is an existing search that we have already carried out. We first conduct a quick, cursory search for our topic of interest. This is known as the 'naive search'. `litsearchr` will then take the results of our naive search and suggest improvements that might capture more articles that are relevant to our topic (and hopefully fewer that are not relevant).

We start by going to the PubMed search page and enter the search in the Query box there. For this example, I entered the following three search terms, combined with AND, like this:

```
(medication) AND (CBT) AND (phobia)
```

If you click on the *Send to* button above the PubMed search results, and then select to send them to *Citation manager*, you will be prompted to download all the results (select "All results" option) to a file that `litsearchr` can read. I have downloaded the example results to a file called `pubmed-medication-set.nbib`.

## Loading and inspection of results

We can load results from a file using the `litsearchr` function `import_results()`. We give the file name as the `file` argument.

```
naive_results <- import_results(file="/Documents/Advanced Course/Lesson materials/Topic 5/pubmed-medication-set.nbib")
```

```
## Reading file /Documents/Advanced Course/Lesson materials/Topic 5/pubmed-medication-set.nbib ... done
```

`import_results()` gives us a dataframe in which each result is a row. We can see from the number of rows how many results our search got.

```
nrow(naive_results)

## [1] 211
```

We can take a look at the first few.

```
naive_results[1:3,]

##                                publication_type language
## 1                   Journal Article and Review      eng
## 2 Journal Article and Meta-Analysis and Review      eng
## 3                   Journal Article and Review      eng
##                          status
## 1 PubMed-not-MEDLINE and epublish
## 2            MEDLINE and ppublish
## 3            MEDLINE and ppublish
##                                                                          author
## 1                                 Wolf TG and Schläppi S and Benz CI and Campus G
## 2 Bandelow B and Reitt M and Röver C and Michaelis S and Görlich Y and Wedekind D
## 3                                      Pelissolo A and Abou Kassm S and Delhay L
##     abstract

1
Hypnosis is a commonly used therapy option in dentistry and medicine for fear and pain
reduction. Nevertheless, it is viewed very critically, as there is still insufficient evidence
for a treatment effect. Specific phobia of dental treatment and dental anxiety are prevalent
conditions that can cause an oral health impairment. This paper critically reviews 19 clinical
trials aimed at reducing dental anxiety and fear avoidance in adults, published in peer-
reviewed journals between 1979 and 2021. The search identified 257 papers; 223 were selected
after removing duplicates. A total of 188 articles were excluded after title and abstract
evaluation; 35 full text articles were assessed for eligibility. Another 10 papers were
discharged after full text evaluation, as these were case reports and questionnaires. Six
papers were discharged due to the lack of a comparable scale to measure dental anxiety. The
following treatment techniques were reviewed: various forms of cognitive-behavioral therapy
(CBT), relaxation training, benzodiazepine premedication, self-hypnosis by audio therapy,
hypnotherapy, hypnosis, and nitrous oxide sedation. CBT delivered in a variety of formats,
including one-session treatment, showed the most evidence for the efficacy of reducing
anxiety. A wide heterogeneity of methods allowed only the inclusion of five studies to the
performed meta-analysis, showing contrasting results for the application of hypnosis. The main
reason for this issue is the great variety in methods used, making a distinct assessment of
hypnotic interventions difficult. However, the results of the systematic review are promising
in that hypnosis can also be regarded as powerful and successful method for anxiety reduction,
while there are also studies with a small or even slightly negative effect. Therefore, further
research is needed. Within the limitations of the current study, a more consistent use of
methods to examine anxiety for hypnosis research is recommended.
```

There are columns for the title, authors, date, abstract, and so on. We can check the names of all the columns to see all the information we have on each search result.

```
colnames(naive_results)

##  [1] "publication_type"              "language"
##  [3] "status"                        "author"
##  [5] "author_full"                   "address"
##  [7] "author_id"                     "date_created"
```

```
##  [9] "date_revised"                  "date_added"
## [11] "date_published_elec"           "date_published"
## [13] "publication_history_status"    "title"
## [15] "journal"                       "journal_abbreviated"
## [17] "source"                        "volume"
## [19] "issue"                         "abstract"
## [21] "mesh_date"                     "article_id"
## [23] "pubmed_id"                     "pubmed_central_identitfier"
## [25] "pubmed_central_release"        "nlm_id"
## [27] "place_published"               "location_id"
## [29] "keywords"                      "term_owner_other"
## [31] "cois"                          "own"
## [33] "date_completed"                "pages"
## [35] "mesh_terms"                    "registry_number"
## [37] "subset"                        "grant_number"
## [39] "cin"                           "ein"
## [41] "title_book"                    "collection_title"
## [43] "PB"                            "copyright_info"
## [45] "personal_name_as_subject"      "personal_name_as_subject_full"
## [47] "uof"                           "references_n"
## [49] "title_transliterated"          "secondary_source_id"
## [51] "manuscript_id"                 "uin"
## [53] "author_corporate"              "abstract_other"
## [55] "oabl"
```

And just as a check, let's take a look at the title of the first result.

```
naive_results[1, "title"]

## [1] "Efficacy of Hypnosis on Dental Anxiety and Phobia: A Systematic
Review and Meta-Analysis."
```

The next step is to analyze the search results that we already have and look to see whether there are more search terms in them that might be related to our topic. If so, we can use these additional terms in a new search to get more relevant results.

There are two different ways of searching for new terms.

## Keywords

Many authors or journals already provide a list of keywords attached to the article. So the simplest way to get new search terms is just to look at what keywords were already provided in the articles we already found.

As an example, we can take a look at the keywords for the first article in our results.

```
naive_results[1, "keywords"]

## [1] "dental anxiety and dental phobia and effect and fear and hypnosis and
hypnotherapy"
```

Some of the keywords are missing (`NA`) from this search. For instance, they are missing from the second article in our results (position depending on the new articles on the topic since this handout was produced).

```
naive_results[2, "keywords"]

## [1] NA
```

How many articles are missing keywords? We can count up the number of `NA` values to find out.

```
sum(is.na(naive_results[, "keywords"]))

## [1] 99
```

Almost half of them! So relying on the provided keywords might not always be such a great approach. But for the purposes of demonstration let's see how we could use them in `litsearchr`.

`litsearchr` has a function `extract_terms()` that can gather the keywords from this column of our search results. The `keywords` argument is where we put the column of keywords from our results dataframe. The `method="tagged"` argument lets `extract_terms()` know that we are getting keywords that article authors themselves have provided (or 'tagged' the article with).

```
extract_terms(keywords=naive_results[, "keywords"], method="tagged")

## Loading required namespace: stopwords

##  [1] "anxiety disorder"
##  [2] "anxiety disorders"
##  [3] "bipolar disorder"
##  [4] "clinical trials"
##  [5] "cognitive behavior therapy"
##  [6] "cognitive behavioral therapy"
##  [7] "cognitive behaviour therapy"
##  [8] "cognitive behavioural therapy"
##  [9] "cognitive bias modification"
## [10] "cognitive therapy"
## [11] "cognitive-behavioral therapy"
## [12] "cognitive-behavioural therapy"
## [13] "commitment therapy"
## [14] "drug treatment"
## [15] "economic evaluation"
## [16] "emotion regulation"
## [17] "emotional disorders"
## [18] "exposure therapy"
## [19] "fear conditioning"
## [20] "fear extinction"
## [21] "fear of childbirth"
## [22] "fear of positive evaluation"
## [23] "functional connectivity"
## [24] "functional magnetic resonance imaging"
## [25] "generalized anxiety disorder"
## [26] "imaging genetics"
## [27] "mental health"
## [28] "mental health services"
## [29] "panic attacks"
## [30] "panic disorder"
## [31] "personality disorder"
## [32] "phobic disorders"
## [33] "physical activity"
## [34] "posttraumatic stress disorder"
## [35] "primary care"
## [36] "psychodynamic therapy"
## [37] "randomized controlled trial"
## [38] "randomized controlled trials"
## [39] "social anxiety"
## [40] "social anxiety disorder"
```

```
## [41] "social phobia"
## [42] "systemic therapy"
## [43] "treatment outcome"
## [44] "unified protocol"
## [45] "virtual reality"
```

We seem to have got only multi-word phrases, no single words. And we didn't get very many. As is often the case in programming, we need to check the documentation for `extract_terms()` to see what its default behavior is. (We can get the documentation in RStudio by typing `? extract_terms` at the console).

We see that `extract_terms()` has a few default arguments:

- `min_freq=2`. Only get keywords that appear at least twice in the full set of results. This is good for making sure that we are only getting keywords that are related to more than just one article in our field of interest. But it might also miss out some important extra suggestions.
- `min_n=2`. Only get keywords that consist of at least two words. This is why we only see multi-word phrases in the keywords we just got.
- `max_n=5`. Get keywords up to five words long. Maybe this is longer than we need.

We can experiment with changing some of these arguments. For example, let's try including single words. This time, let's store the result in a variable, as we will use it later on.

```
keywords <- extract_terms(keywords=naive_results[, "keywords"],
method="tagged", min_n=1)
```

```
keywords
```

```
##  [1] "acceptance"
##  [2] "adherence"
##  [3] "adolescent"
##  [4] "adolescents"
##  [5] "agoraphobia"
##  [6] "antidepressant"
##  [7] "antidepressants"
##  [8] "anxiety"
##  [9] "anxiety disorder"
## [10] "anxiety disorders"
## [11] "autism"
## [12] "bipolar disorder"
## [13] "cbt"
## [14] "child"
## [15] "children"
## [16] "clinical trials"
## [17] "cognitive behavior therapy"
## [18] "cognitive behavioral therapy"
## [19] "cognitive behaviour therapy"
## [20] "cognitive behavioural therapy"
## [21] "cognitive bias modification"
## [22] "cognitive therapy"
## [23] "cognitive-behavioral therapy"
## [24] "cognitive-behavioural therapy"
## [25] "commitment therapy"
## [26] "comorbidity"
## [27] "cost-effectiveness"
## [28] "d-cycloserine"
```

```
## [29] "depression"
## [30] "drug treatment"
## [31] "economic evaluation"
## [32] "ehealth"
## [33] "emotion regulation"
## [34] "emotional disorders"
## [35] "exercise"
## [36] "exposure"
## [37] "exposure therapy"
## [38] "extinction"
## [39] "fear"
## [40] "fear conditioning"
## [41] "fear extinction"
## [42] "fear of childbirth"
## [43] "fear of positive evaluation"
## [44] "fmri"
## [45] "functional connectivity"
## [46] "functional magnetic resonance imaging"
## [47] "generalized anxiety disorder"
## [48] "genetics"
## [49] "guideline"
## [50] "imaging genetics"
## [51] "manual"
## [52] "mental health"
## [53] "mental health services"
## [54] "meta-analysis"
## [55] "misophonia"
## [56] "outcome"
## [57] "panic"
## [58] "panic attacks"
## [59] "panic disorder"
## [60] "pediatric"
## [61] "personality disorder"
## [62] "pharmacotherapy"
## [63] "phobia"
## [64] "phobic disorders"
## [65] "physical activity"
## [66] "posttraumatic stress disorder"
## [67] "predictors"
## [68] "pregnancy"
## [69] "primary care"
## [70] "psychoanalytic"
## [71] "psychodynamic therapy"
## [72] "psychotherapy"
## [73] "randomized controlled trial"
## [74] "randomized controlled trials"
## [75] "self-management"
## [76] "social anxiety"
## [77] "social anxiety disorder"
## [78] "social phobia"
## [79] "systemic therapy"
## [80] "therapygenetics"
## [81] "treatment"
## [82] "treatment outcome"
## [83] "unified protocol"
## [84] "virtual reality"
```

This gets us more search terms. Some of these might be useful new terms to include in our literature search. But others are clearly too broad, for example outcome, or are from tangential topics, for example virtual reality.

You could try narrowing the terms down a bit, for example by asking only for those that are provided for at least three of the articles in our search. Where there are additional

arguments to a function like this, it is a good idea to try out a few variations to get some idea of how they work. You can give this a go. For now, let's move on to a different method of getting search terms from our naive search.

## Titles and abstracts

We saw above that not all articles even provide keywords. And maybe the authors of the articles themselves can't always be relied upon to tag their articles with all the keywords that might link their article to relevant topics. So an alternative (or supplementary) method of finding new relevant search terms is instead to search in the titles and/or abstracts of the articles.

Here we will search in the titles only, to keep the example manageable. The article abstracts are much longer and contain many additional irrelevant words. Filtering these out would take a lot of work.

Titles are fairly short and should contain mostly relevant terms. But we will still need to filter them down a bit to get only the 'interesting' words. Somebody has already invented a method for doing this, called Rapid Automatic Keyword Extraction (RAKE). `litsearchr` can apply this method if we tell it to (the argument for doing so has the curious name `"fakerake"`, because in fact `litsearchr` uses a slightly simplified version of the full RAKE technique).

Just as when we searched among the author-provided keywords, here too we can give arguments `min_n`, and `max_n` to choose whether we include single words or multi-word phrases, and we can give `min_freq` to exclude words that occur in too few of the titles in our original search.

```
extract_terms(text=naive_results[, "title"], method="fakerake", min_freq=3,
min_n=2)

##  [1] "anxiety disorder"
##  [2] "anxiety disorders"
##  [3] "anxiety symptoms"
##  [4] "anxious children"
##  [5] "behavior therapy"
##  [6] "behavioral therapy"
##  [7] "behavioral therapy response"
##  [8] "behaviour therapy"
##  [9] "behavioural therapy"
## [10] "bipolar disorder"
## [11] "child anxiety"
## [12] "childhood anxiety"
## [13] "childhood anxiety disorders"
## [14] "clinical practice"
## [15] "clinical trial"
## [16] "cognitive behavior"
## [17] "cognitive behavior therapy"
## [18] "cognitive behavioral"
## [19] "cognitive behavioral therapy"
## [20] "cognitive behavioral therapy response"
## [21] "cognitive behaviour"
## [22] "cognitive behaviour therapy"
## [23] "cognitive behavioural"
## [24] "cognitive behavioural therapy"
## [25] "cognitive-behavior therapy"
## [26] "cognitive-behavioral therapy"
```

```
## [27] "cognitive-behavioral treatment"
## [28] "commitment therapy"
## [29] "controlled pilot"
## [30] "controlled pilot trial"
## [31] "controlled trial"
## [32] "emotion regulation"
## [33] "generalized social"
## [34] "generalized social anxiety"
## [35] "generalized social anxiety disorder"
## [36] "group cognitive"
## [37] "group cognitive behavioral"
## [38] "group cognitive behavioral therapy"
## [39] "group therapy"
## [40] "longitudinal study"
## [41] "neural correlates"
## [42] "one-session treatment"
## [43] "panic disorder"
## [44] "pediatric anxiety"
## [45] "pediatric anxiety disorders"
## [46] "pilot study"
## [47] "pilot trial"
## [48] "predict response"
## [49] "psychodynamic therapy"
## [50] "randomised controlled"
## [51] "randomized clinical"
## [52] "randomized clinical trial"
## [53] "randomized controlled"
## [54] "randomized controlled pilot"
## [55] "randomized controlled pilot trial"
## [56] "randomized controlled trial"
## [57] "social anxiety"
## [58] "social anxiety disorder"
## [59] "social phobia"
## [60] "specific phobias"
## [61] "study protocol"
## [62] "systematic review"
## [63] "therapy response"
## [64] "therapy versus"
## [65] "treatment outcome"
## [66] "treatment response"
## [67] "virtual reality"
## [68] "young people"
```

Searching in the titles gets us more search terms than we got from just the keywords provided by the authors when we choose `min_n=2`. In part this is because every article has a title whereas not every article supplies keywords. And in part it is because the titles also contain additional irrelevant words, whereas the keywords have been specially chosen for their relevance. The title search probably gets us many more terms than we need.

Some of the phrases that appear in a title are general science or data analysis terms and are not related to the specific topic of the article. In language analysis, such frequently-occurring but uninformative words are often called 'stopwords'. If we are going to work with `litsearchr` a lot and will often need to filter out the same set of stopwords, then it can be handy to keep these words in a text file. We can then read this file into R when we need them, and we can add to the file each time we encounter more words that we know are irrelevant.

This example contains a (non-exhaustive) list of generic clinical psychology words that occurred in this set of articles. They are stored in the file *clin_psy_stopwords.txt*. Let's read it in and take a look.

```r
clinpsy_stopwords <- read_lines("/Users/panj/Documents/ISADS/Workshops and
seminars/Advanced Course/Lesson materials/Topic 5/clin_psy_stopwords.txt")
clinpsy_stopwords
```

```
##   [1] "advances"        "analyse"         "analysed"        "analyses"
##   [5] "analysing"       "analysis"        "analyze"         "analyzes"
##   [9] "analyzed"        "analyzing"       "assess"          "assessed"
##  [13] "assesses"        "assessing"       "assessment"      "assessments"
##  [17] "benefit"         "based"           "benefits"        "change"
##  [21] "changed"         "changes"         "changing"        "characteristic"
##  [25] "characteristics" "characterize"    "characterized"   "characterizes"
##  [29] "characterizing"  "clinical"        "cluster"         "combine"
##  [33] "combined"        "combines"        "combining"       "comorbid"
##  [37] "comorbidity"     "compare"         "compared"        "compares"
##  [41] "comparing"       "comparison"      "control"         "controlled"
##  [45] "controlling"     "controls"        "design"          "designed"
##  [49] "designing"       "effect"          "effective"       "effectiveness"
##  [53] "effects"         "efficacy"        "feasible"        "feasibility"
##  [57] "follow"          "followed"        "following"       "follows"
##  [61] "group"           "groups"          "impact"          "intervention"
##  [65] "interventions"   "longitudinal"    "moderate"        "moderated"
##  [69] "moderates"       "moderating"      "moderator"       "moderators"
##  [73] "outcome"         "outcomes"        "patient"         "patients"
##  [77] "people"          "pilot"           "practice"        "predict"
##  [81] "predicted"       "predicting"      "predictor"       "predictors"
##  [85] "predicts"        "preliminary"     "primary"         "protocol"
##  [89] "quality"         "random"          "randomise"       "randomised"
##  [93] "randomising"     "randomize"       "randomized"      "randomizing"
##  [97] "rationale"       "reduce"          "reduced"         "reduces"
## [101] "reducing"        "related"         "report"          "reported"
## [105] "reporting"       "reports"         "response"        "responses"
## [109] "result"          "resulted"        "resulting"       "results"
## [113] "review"          "studied"         "studies"         "study"
## [117] "studying"        "systematic"      "treat"           "treated"
## [121] "treating"        "treatment"       "treatments"      "treats"
## [125] "trial"           "trials"          "versus"
```

The `extract_terms()` function provides a `stopwords` argument that we can use to filter out words. `litsearchr` also provides a general list of stopwords for English (and for some other languages) via the `get_stopwords()` function, so we can also add this to our own more specific stopwords.

```r
all_stopwords <- c(get_stopwords("English"), clinpsy_stopwords)
```

Now let's use this big list of stopwords to have another go at extracting only relevant search terms.

```r
title_terms <- extract_terms(
  text=naive_results[, "title"],
  method="fakerake",
  min_freq=3, min_n=2,
  stopwords=all_stopwords
)
```

```
title_terms
```

```
##  [1] "anxiety disorder"                    "anxiety disorders"
##  [3] "anxiety symptoms"                    "anxious children"
##  [5] "behavior therapy"                    "behavioral therapy"
##  [7] "behaviour therapy"                   "behavioural therapy"
```

```
##  [9] "bipolar disorder"                    "child anxiety"
## [11] "childhood anxiety"                   "childhood anxiety disorders"
## [13] "cognitive behavior"                  "cognitive behavior therapy"
## [15] "cognitive behavioral"                "cognitive behavioral therapy"
## [17] "cognitive behaviour"                 "cognitive behaviour therapy"
## [19] "cognitive behavioural"               "cognitive behavioural therapy"
## [21] "cognitive-behavior therapy"          "cognitive-behavioral therapy"
## [23] "commitment therapy"                  "emotion regulation"
## [25] "generalized social"                  "generalized social anxiety"
## [27] "generalized social anxiety disorder" "neural correlates"
## [29] "panic disorder"                      "pediatric anxiety"
## [31] "pediatric anxiety disorders"         "psychodynamic therapy"
## [33] "social anxiety"                      "social anxiety disorder"
## [35] "social phobia"                       "specific phobias"
## [37] "virtual reality"
```

This looks a bit better. We should be careful not to accidentally exclude any potentially relevant words at this stage, and as usual exploration is important. Exercise: Try out different lists of stopwords and different values of `min_freq`, `min_n`, and `max_n` to get a clear impression of the articles in the naive search.

Let's finish by adding together the search terms we got from the article titles and those we got from the keywords earlier, removing duplicates.

```
terms <- unique(c(keywords, title_terms))
```

## Network analysis

Our list of new search terms is looking fairly good. But there are probably still some in there that are unrelated to the others and to our topic of interest. These perhaps only occur in a small number of articles that do not mention many of the other search terms. We would like some systematic way of identifying these 'isolated' search terms.

One way to do this is to analyze the search terms as a network. The idea behind this is that terms are linked to each other by virtue of appearing in the same articles. If we can find out which terms tend to occur together in the same article, we can pick out groups of terms that are probably all referring to the same topic (because they occur in the same subset of articles), and we can filter out terms that do not often occur together with any of the main groups of terms.

We don't have the full texts of the articles, and in any case checking for search terms in a large number of full texts would be a big computational task. So we will take the title and abstract of each article as the 'content' of that article, and count a term as having occurred in that article if it is to be found in either the title or abstract. For this, we need to join the title of each article to its abstract.

```
docs <- paste(naive_results[, "title"], naive_results[, "abstract"])
```

Let's just check the first one to make sure we did this right.

```
docs[1]
```

```
## [1] "Efficacy of Hypnosis on Dental Anxiety and Phobia: A Systematic Review and Meta-Analysis. Hypnosis is a commonly used therapy option in dentistry and medicine for fear and pain reduction. Nevertheless, it is viewed very critically, as there is still insufficient
```

```
evidence for a treatment effect. Specific phobia of dental treatment and dental anxiety are
prevalent conditions that can cause an oral health impairment. This paper critically reviews
19 clinical trials aimed at reducing dental anxiety and fear avoidance in adults, published in
peer-reviewed journals between 1979 and 2021. The search identified 257 papers; 223 were
selected after removing duplicates. A total of 188 articles were excluded after title and
abstract evaluation; 35 full text articles were assessed for eligibility. Another 10 papers
were discharged after full text evaluation, as these were case reports and questionnaires. Six
papers were discharged due to the lack of a comparable scale to measure dental anxiety. The
following treatment techniques were reviewed: various forms of cognitive-behavioral therapy
(CBT), relaxation training, benzodiazepine premedication, self-hypnosis by audio therapy,
hypnotherapy, hypnosis, and nitrous oxide sedation. CBT delivered in a variety of formats,
including one-session treatment, showed the most evidence for the efficacy of reducing
anxiety. A wide heterogeneity of methods allowed only the inclusion of five studies to the
performed meta-analysis, showing contrasting results for the application of hypnosis. The main
reason for this issue is the great variety in methods used, making a distinct assessment of
hypnotic interventions difficult. However, the results of the systematic review are promising
in that hypnosis can also be regarded as powerful and successful method for anxiety reduction,
while there are also studies with a small or even slightly negative effect. Therefore, further
research is needed. Within the limitations of the current study, a more consistent use of
methods to examine anxiety for hypnosis research is recommended."
```

We now create a matrix that records which terms appear in which articles. The `litsearchr` function `create_dfm()` does this. 'DFM' stands for 'document-feature matrix', where the 'documents' are our articles and the 'features' are the search terms. The `elements` argument is the list of documents. The `features` argument is the list of terms whose relationships we want to analyze within that set of documents.

```
dfm <- create_dfm(elements=docs, features=terms)
```

The rows of our matrix represent the articles (their titles and abstracts), and the columns represent the search terms. Each entry in the matrix records how many times that article contains that term. For example, if we look at the first three articles we see that *acceptance* and *adherence* does not occur in any of them, *adolescent* occurs in the fifth, *antidepressant* occurs in the second and fourth.

```
dfm[1:5, 1:7]

##      acceptance adherence adolescent adolescents agoraphobia antidepressant
## [1,]          0         0          0           0           0              0
## [2,]          0         0          0           0           0              1
## [3,]          0         0          0           0           0              0
## [4,]          0         0          0           0           0              1
## [5,]          0         0          1           1           0              0
##      antidepressants
## [1,]               0
## [2,]               1
## [3,]               0
## [4,]               1
## [5,]               0
```

We can then turn this matrix into a network of linked search terms, using the `litsearchr` function `create_network()`. This function has an argument `min_studies` that excludes terms that occur in fewer than a given number of articles.

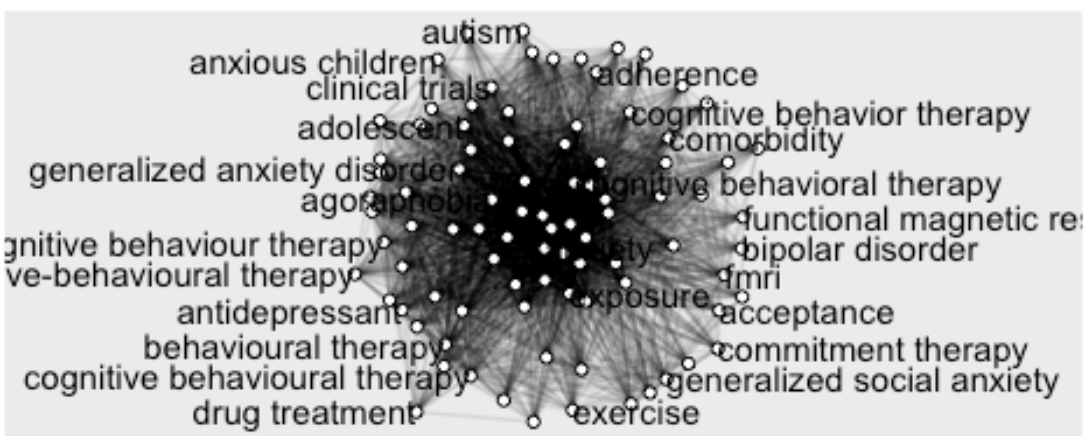
```
g <- create_network(dfm, min_studies=3)
```

## ggraph

We should try to make a picture of our network of terms to get a better idea of its structure. This isn't easy, as we need to show a large number of terms plus all the links between them. The `ggraph` package provides some great tools for drawing networks. There is a lot to learn and discover about drawing networks, and you can look at the tutorials for `ggraph` if you want to learn more. But here network visualization is just an aside, so we will keep it simple.

The `ggraph()` function takes the network that we got from `create_network()` as its argument and draws it as a graph. In addition, we can specify a layout for the graph. Here we use the 'Kamada and Kawai' layout. To sum it up very simply, this layout draws terms that are closely linked close together, and those that are less closely linked further away from each other. We add some labels showing what the actual terms are, using `geom_node_text()`. Since there are far too many terms to show all of them without completely cluttering the figure, we show just an arbitrary subset of them that do not overlap on the figure, using the argument `check_overlap=TRUE`. Finally, we also add lines linking the terms, using `geom_edge_link()`. We color these lines so there are more solid lines linking terms that appear in more articles together (this is called the 'weight' of the link).

```
ggraph(g, layout="stress") +
  coord_fixed() +
  expand_limits(x=c(-3, 3)) +
  geom_edge_link(aes(alpha=weight)) +
  geom_node_point(shape="circle filled", fill="white") +
  geom_node_text(aes(label=name), hjust="outward", check_overlap=TRUE) +
  guides(edge_alpha=FALSE)
```




Given the way in which we drew the graph, it tells us a few things about our search terms.

Terms that appear near the center of the graph and that are linked to each other by darker lines are probably more important for our overall topic. Here, these include, for example, exposure, anxiety disorder, and cognitive behavioral therapy.

Terms that appear at the periphery of the graph and linked to it only by faint lines are not closely related to any other terms. These are mostly tangential terms that are related to, but not part of, our main topic, for example autism, anxious children, and drug treatment.

Interestingly, we see some anomalies at the periphery of the graph. The clearly very relevant terms cognitive behavior therapy and cognitive behavioural therapy appear far from the center despite being variants of one of our main naive search terms. This is probably because they are minority variations on the terminology. One involves the British spelling of behaviour, and the other uses the word behavior where most authors prefer behavioral. We will need to treat the British spelling issue as a special case later on, and this is a good illustration of the importance of looking carefully at our results.

## Pruning

Now let's use the network to rank our search terms by importance, with the aim of pruning away some of the least important ones.

The 'strength' of each term in the network is the number of other terms that it appears together with. We can get this information from our network using the `strength()` function from the `igraph` package (behind the scenes, `litsearchr` uses `igraph` for some of the workings of its network analyses). If we then arrange the terms in ascending order of strength we see those that might be the least important.

```
strengths <- strength(g)

data.frame(term=names(strengths), strength=strengths, row.names=NULL) %>%
  mutate(rank=rank(strength, ties.method="min")) %>%
  arrange(strength) ->
  term_strengths
```

```
term_strengths
```

```
##                                    term strength rank
## 1            cognitive bias modification       33    1
## 2                    emotional disorders       36    2
## 3                                 autism       38    3
## 4                       bipolar disorder       42    4
## 5                         drug treatment       44    5
## 6                       systemic therapy       44    5
## 7                   personality disorder       50    7
## 8                functional connectivity       53    8
## 9                     commitment therapy       55    9
## 10                     neural correlates       64   10
## 11                    emotion regulation       70   11
## 12                            acceptance       72   12
## 13          randomized controlled trials       75   13
## 14                        panic attacks       76   14
## 15                       fear extinction       77   15
## 16                      anxious children       77   15
## 17                     cognitive therapy       78   17
## 18                              exercise       80   18
## 19            cognitive-behavior therapy       80   18
## 20                 psychodynamic therapy       88   20
## 21           pediatric anxiety disorders       97   21
## 22         cognitive-behavioural therapy       98   22
## 23                             adherence      101   23
## 24                       virtual reality      101   23
```

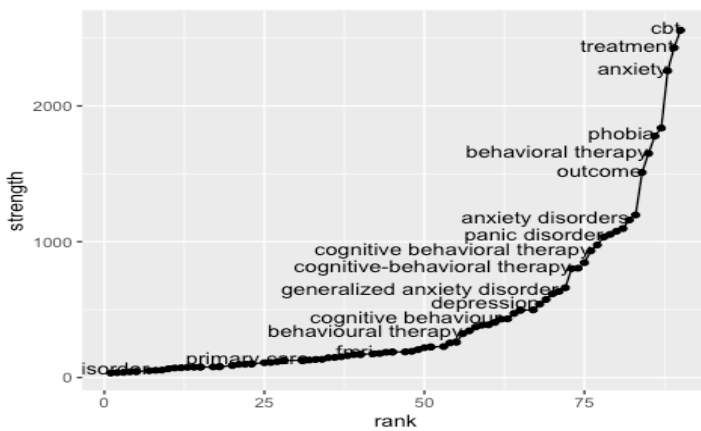
```
## 25         posttraumatic stress disorder      109   25
## 26                         child anxiety      111   26
## 27 functional magnetic resonance imaging      117   27
## 28                    generalized social      124   28
## 29            generalized social anxiety      124   28
## 30   generalized social anxiety disorder      124   28
## 31                     fear conditioning      128   31
## 32                          primary care      129   32
## 33                   cost-effectiveness      133   33
## 34                     exposure therapy      134   34
## 35                          comorbidity      146   35
## 36            cognitive behavior therapy      148   36
## 37                       clinical trials      153   37
## 38                            extinction      161   38
## 39                       antidepressants      169   39
## 40           cognitive behaviour therapy      170   40
## 41                         d-cycloserine      170   40
## 42                                  fmri      175   42
## 43           childhood anxiety disorders      177   43
## 44                     childhood anxiety      186   44
## 45                        antidepressant      188   45
## 46                     pediatric anxiety      188   45
## 47                         meta-analysis      189   47
## 48                     behaviour therapy      193   48
## 49                      specific phobias      205   49
## 50                             pediatric      220   50
## 51         cognitive behavioural therapy      225   51
## 52                         mental health      225   51
## 53                      behavior therapy      228   53
## 54                             guideline      255   54
## 55                 cognitive behavioural      260   55
## 56                   behavioural therapy      323   56
## 57                       pharmacotherapy      344   57
## 58           randomized controlled trial      372   58
## 59                                 manual      386   59
## 60                           adolescents      388   60
## 61                            predictors      406   61
## 62                   cognitive behaviour      430   62
## 63                         psychotherapy      432   63
## 64                     treatment outcome      472   64
## 65                                  fear      496   65
## 66                      anxiety symptoms      496   65
## 67                             adolescent      498   67
## 68                             depression      540   68
## 69                               exposure      575   69
## 70                           agoraphobia      616   70
## 71          generalized anxiety disorder      634   71
## 72                               children      661   72
## 73          cognitive-behavioral therapy      802   73
## 74                                  child      805   74
## 75                         social phobia      846   75
## 76          cognitive behavioral therapy      933   76
## 77                  cognitive behavioral      976   77
## 78                        panic disorder     1034   78
## 79                social anxiety disorder     1054   79
## 80                                  panic     1077   80
## 81                    cognitive behavior     1097   81
## 82                     anxiety disorders     1160   82
## 83                        social anxiety     1197   83
## 84                               outcome     1510   84
## 85                    behavioral therapy     1650   85
## 86                                 phobia     1778   86
## 87                      anxiety disorder     1837   87
## 88                               anxiety     2259   88
```

```
## 89                          treatment     2428   89
## 90                                cbt     2557   90
```

At the top are the terms that are most weakly linked to the others. For some of them you can compare this with their positions on the graph visualization above, where they appear near the margins of the figure. In most cases, terms like these are completely irrelevant and have occurred in a few of the articles in the naive search for arbitrary reasons. Some are perhaps still relevant but are just very rarely used.

We would like to discard some of the terms that only rarely occur together with the others. What rule can we use to make a decision about which to discard? To get an idea of how we might approach this question, let's visualize the strengths of the terms.

```
cutoff_fig <- ggplot(term_strengths, aes(x=rank, y=strength, label=term)) +
  geom_line() +
  geom_point() +
  geom_text(data=filter(term_strengths, rank>5), hjust="right", nudge_y=20,
check_overlap=TRUE)

cutoff_fig
```




The figure shows the terms in ascending order of strength from left to right. Again, only an arbitrary subset of the terms are labeled, so as not to clutter the figure. We can see for example that there are five terms (starting with cbt (cognitive behavioral therapy)) that have much higher strengths than the others.

Let's use this figure to visualize two methods that `litsearchr` offers for pruning away the search terms least closely linked to the others. Both of these involve finding a cutoff value for term strength, such that we discard terms with a strength below that value. The find_cutoff() function implements these methods.

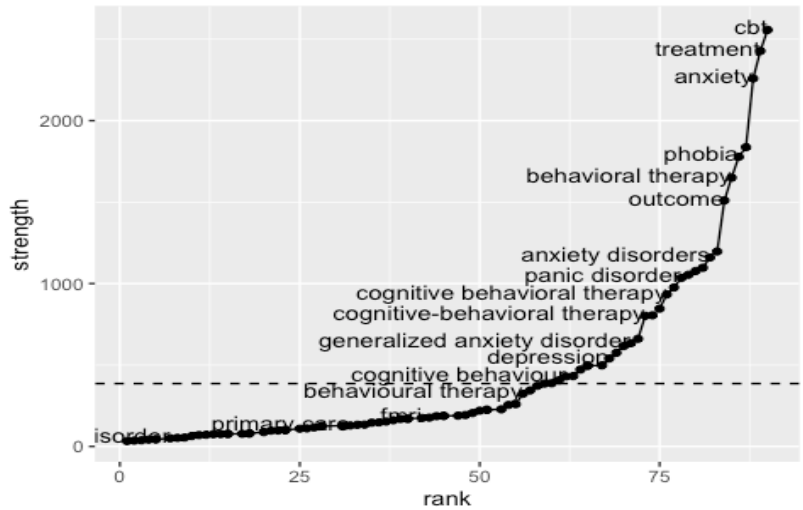
### Cumulatively

One simple way to decide on a cutoff is to choose to retain a certain proportion of the total strength of the network of search terms, for example 80%. If we supply the argument `method="cumulative` to the `find_cutoff()` function, we get the cutoff strength value according to this method. The `percent` argument specifies what proportion of the total strength we would like to retain.

```
cutoff_cum <- find_cutoff(g, method="cumulative", percent=0.8)

cutoff_cum

## [1] 386
```

Let's see this on our figure.

```
cutoff_fig +
  geom_hline(yintercept=cutoff_cum, linetype="dashed")
```




Once we have found a cutoff value, the `reduce_graph()` function applies it and prunes away the terms with low strength. The arguments are the original network and the cutoff. The `get_keywords()` function then gets the remaining terms from the reduced network.

```
get_keywords(reduce_graph(g, cutoff_cum))

##  [1] "adolescent"                   "adolescents"
##  [3] "agoraphobia"                  "anxiety"
##  [5] "anxiety disorder"             "anxiety disorders"
##  [7] "cbt"                          "child"
##  [9] "children"                     "cognitive behavioral therapy"
## [11] "cognitive-behavioral therapy" "depression"
## [13] "exposure"                     "fear"
## [15] "generalized anxiety disorder" "manual"
## [17] "outcome"                      "panic"
## [19] "panic disorder"               "phobia"
## [21] "predictors"                   "psychotherapy"
## [23] "social anxiety"               "social anxiety disorder"
## [25] "social phobia"                "treatment"
## [27] "treatment outcome"            "anxiety symptoms"
## [29] "behavioral therapy"           "cognitive behavior"
## [31] "cognitive behavioral"         "cognitive behaviour"
```

### Changepoints

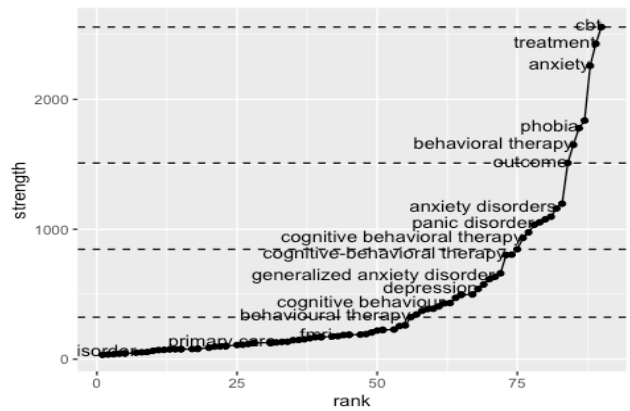
Looking at the figure above, another method of pruning away terms suggests itself. There are certain points along the ranking of terms where the strength of the next strongest term is much greater than that of the previous one (places where the ascending line 'jumps up'). We could use these places as cutoffs, since the terms below them have much lower strength

than those above. There may of course be more than one place where term strength jumps up like this, so we will have multiple candidates for cutoffs. The same `find_cutoff()` function with the argument `method="changepoint"` will find these cutoffs. The `knot_num` argument specifies how many 'knots' we wish to slice the keywords into.

```
cutoff_change <- find_cutoff(g, method="changepoint", knot_num=3)
cutoff_change

## [1]  323  846 1510 2557
```

This time we get several suggested cutoffs. Let's put them on our figure, where we can see that they cut off the search terms just before large increases in term strength.

```
cutoff_fig +
  geom_hline(yintercept=cutoff_change, linetype="dashed")
```




After doing this, we can apply the same `reduce_graph()` function that we did for the cumulative strength method. The only difference is that we have to pick one of the cutoffs in our vector.

```
g_redux <- reduce_graph(g, cutoff_change[1])
selected_terms <- get_keywords(g_redux)
selected_terms

##  [1] "adolescent"                   "adolescents"
##  [3] "agoraphobia"                  "anxiety"
##  [5] "anxiety disorder"             "anxiety disorders"
##  [7] "cbt"                          "child"
##  [9] "children"                     "cognitive behavioral therapy"
## [11] "cognitive-behavioral therapy" "depression"
## [13] "exposure"                     "fear"
## [15] "generalized anxiety disorder" "manual"
## [17] "outcome"                      "panic"
## [19] "panic disorder"               "pharmacotherapy"
## [21] "phobia"                       "predictors"
## [23] "psychotherapy"                "randomized controlled trial"
## [25] "social anxiety"               "social anxiety disorder"
## [27] "social phobia"                "treatment"
## [29] "treatment outcome"            "anxiety symptoms"
## [31] "behavioral therapy"           "behavioural therapy"
## [33] "cognitive behavior"           "cognitive behavioral"
## [35] "cognitive behaviour"
```

Remember the issue with the British spelling of *behaviour*. We should add this term back in. It is also probably a good idea to add back in the terms of our original naive search if they did not turn up in our final set of terms.

```
extra_terms <- c(
  "medication",
  "cognitive-behavioural therapy",
  "cognitive behavioural therapy"
)

selected_terms <- c(selected_terms, extra_terms)

selected_terms
```

```
##  [1] "adolescent"                    "adolescents"
##  [3] "agoraphobia"                   "anxiety"
##  [5] "anxiety disorder"              "anxiety disorders"
##  [7] "cbt"                           "child"
##  [9] "children"                      "cognitive behavioral therapy"
## [11] "cognitive-behavioral therapy"  "depression"
## [13] "exposure"                      "fear"
## [15] "generalized anxiety disorder"  "manual"
## [17] "outcome"                       "panic"
## [19] "panic disorder"                "pharmacotherapy"
## [21] "phobia"                        "predictors"
## [23] "psychotherapy"                 "randomized controlled trial"
## [25] "social anxiety"                "social anxiety disorder"
## [27] "social phobia"                 "treatment"
## [29] "treatment outcome"             "anxiety symptoms"
## [31] "behavioral therapy"            "behavioural therapy"
## [33] "cognitive behavior"            "cognitive behavioral"
## [35] "cognitive behaviour"           "medication"
## [37] "cognitive-behavioural therapy" "cognitive behavioural therapy"
```

## Grouping

Now that we have got a revised list of search terms from the results of our naive search, we want to turn them into a new search query that we can use to get more articles relevant to the same topic. For this new, hopefully more rigorous, search we will need a combination of OR and AND operators. The OR operator should combine search terms that are all about the same subtopic, so that we get articles that contain any one of them. The AND operator should combine these groups of search terms so that we get only articles that mention at least one term from each of the subtopics that we are interested in.

In our starting example we had three subtopics, medication, cognitive-behavioral therapy, and phobias. So the first step is to take the terms that we have found and group them into clusters related to each of these topics. (Or it may be the case that our work with `litsearchr` has led us to rethink our subtopics or to add new ones.)

There are methods for automatically grouping networks into clusters, but these are not always so reliable; computers don't understand what words mean. Currently the `litsearchr` documentation recommends doing this step manually. We look at our search terms, and put them into a list of separate vectors, one for each subtopic.

```r
grouped_terms <-list(
  medication=selected_terms[c(20, 36)],
  cbt=selected_terms[c(7, 10, 11, 31, 32, 33, 34, 35, 37, 38)],
  phobia=selected_terms[c(4, 5, 6, 14, 15, 18, 19, 21, 25, 30)]
)

grouped_terms
```

```
## $medication
## [1] "pharmacotherapy" "medication"
##
## $cbt
##  [1] "cbt"                           "cognitive behavioral therapy"
##  [3] "cognitive-behavioral therapy"  "behavioral therapy"
##  [5] "behavioural therapy"           "cognitive behavior"
##  [7] "cognitive behavioral"          "cognitive behaviour"
##  [9] "cognitive-behavioural therapy" "cognitive behavioural therapy"
##
## $phobia
##  [1] "anxiety"                      "anxiety disorder"
##  [3] "anxiety disorders"            "fear"
##  [5] "generalized anxiety disorder" "panic"
##  [7] "panic disorder"               "phobia"
##  [9] "social anxiety"               "anxiety symptoms"
```

## Writing a new search

The `write_search()` function takes our list of grouped search terms and writes the text of a new search. There are quite a few arguments to take care of for this function:

- `languages` provides a list of languages to translate the search into, in case we want to get articles in multiple languages.
- `exactphrase` controls whether terms that consist of more than one word should be matched exactly rather than as two separate words. If we have phrases that are only relevant as a whole phrase, then we should set this to `TRUE`, so that for example social phobia will not also catch all the articles containing the word social.
- `stemming` controls whether words are stripped down to the smallest meaningful part of the word (its 'stem') so that we make sure to catch all variants of the word, for example catching both *behavior** and *behavioral**.
- `closure` controls whether partial matches are matched at the left end of a word ("left"), at the right ("right"), only as exact matches ("full") or as any word containing a term ("none").
- `writesearch` controls whether we would like to write the search text to a file.

```r
write_search(
  grouped_terms,
  languages="English",
  exactphrase=TRUE,
  stemming=FALSE,
  closure="left",
  writesearch=TRUE
)
```

Let's read in the contents of the text file that we just wrote, to see what our search text looks like.

```
cat(read_file("search-inEnglish.txt"))

## \(\(pharmacotherapy OR medication\) AND \(cbt OR "cognitive-behavioral therapy" OR "behavioral therapy" OR "behavioural therapy" OR "cognitive behavior" OR "cognitive behaviour" OR "cognitive-behavioural therapy"\) AND \(anxiety OR fear OR "generalized anxiety disorder" OR panic OR phobia OR "social anxiety"\)\)
```

We can now go back to the search site and copy the contents of this text file into the search field to conduct a new search.

## Checking the new search

I ran our new search and downloaded the results to a file called **pubmed-pharmacoth-set.nbib**. Let's load this file with `litsearchr`. This may take a moment, as this file contains a lot more results than the naive search.

```
new_results <- import_results(file="/Documents/Advanced Course/Lesson materials/Topic 5/pubmed-pharmacoth-set.nbib")

## Reading file /Documents/Advanced Course/Lesson materials/Topic 5/pubmed-pharmacoth-set.nbib ... done
```

How many did we get?

```
nrow(new_results)

## [1] 5691
```

We now need to check whether the new results seem to be relevant to our chosen topic. There are a few basic things that we can check.

### Against the naive search

We can first check whether all of the results of the naive search are in the new search. Since we conducted the naive search using the most important terms that occurred to us for our topic, and since we included these same terms or very similar in our new search, we ought to get the same articles back among our new results, at least if they were really relevant.

```
naive_results %>%
  mutate(in_new_results=title %in% new_results[, "title"]) ->
  naive_results

naive_results %>%
  filter(!in_new_results) %>%
  select(title, keywords)

## [1] title    keywords
## <0 rows> (or 0-length row.names)
```

We didn't miss any of the articles from the naive search. Good.

### Against gold standard results

If we have started reviewing the literature on our chosen topic, we may already know the titles of some important articles that have been written on the subject. So another way of checking our new search is to check whether it includes specific important results. `litsearchr` provides a function called `check_recall()` for searching for specific titles. We do not have to get the titles of our desired articles exactly right. If the capitalization or punctuation is slightly different from the version in our results dataframe, `check_recall()` will find the closest match.

Let's try it.

```
important_titles <- c(
  "Efficacy of treatments for anxiety disorder: A meta-analysis",
  "Cognitive behaviour therapy for health anxiety: A systematic review and
meta-analysis",
  "A systematic review and meta-analysis of treatments for agrophobia"
)

data.frame(check_recall(important_titles, new_results[, "title"]))
```

```
##                                                                                  Title
## 1                          Efficacy of treatments for anxiety disorder: A meta-analysis
## 2 Cognitive behaviour therapy for health anxiety: A systematic review and meta-analysis
## 3                    A systematic review and meta-analysis of treatments for agrophobia
##
Best_Match
## 1                                                                        Efficacy of
treatments for anxiety disorders: a meta-analysis.
## 2                                                  Cognitive Behaviour Therapy for Health
Anxiety: A Systematic Review and Meta-Analysis.
## 3 A systematic review and meta-analysis of Internet-based self-help interventions for
mental health among adolescents and college students.
##          Similarity
## 1 0.752136752136752
## 2                 1
## 3 0.424242424242424
```

All of the best matches in the output table are clearly the articles that we were looking for, so our new search has found these.

You can try to refine the example search further. We will run our case studies during the topic's group projects.

# PART II: Facilitating systematic reviews, data extraction and meta-analysis

## Introduction

Critical goals to research synthesis practices are to generate robust, aggregate views of accumulated research findings, and to explore sources of variability in that research. Ideally, these goals are achieved by pairing systematic review methods with the statistics of

meta-analysis to yield high-quality syntheses (Lajeunesse, 2016). For example, a systematic review is first used to carefully plan and implement a strategy to locate studies, assess their eligibility and extract relevant research findings in a repeatable and transparent way. A meta-analysis is then used to statistically model sources of variability within and between studies with the aim to quantitatively weight and aggregate their findings (Lajeunesse, 2016). Although this taxonomy of practices is meant to produce scientifically explicit and defensible overviews of research, it is also accompanied by a variety of implementation challenges. For example, the number and complexity of tasks involved in generating a high-quality synthesis require a diverse combination of software tools – which are numerous and growing due to broad interdisciplinary interests in research synthesis. This diversity of tools can reduce efforts to make synthesis more repeatable and transparent, given that it can yield mixed standards for solving common problems with the screening, extraction and analysis of study outcomes (Lajeunesse, 2016). Part II uses the `metagear` package developed by Lajeunesse (2016) for `r`. It is a comprehensive, multifunctional toolbox with capabilities aimed to cover much of the research synthesis taxonomy: from applying a systematic review approach to objectively assemble and screen the literature, to extracting data from studies, and to finally summarize and analyse these data with the statistics of meta-analysis. We will explore two of the current functionalities of `metagear`: an abstract screener GUI to efficiently sieve bibliographic information from large numbers of candidate studies; tools to assign screening effort across multiple collaborators/reviewers and to assess inter-reviewer reliability using kappa statistics; PDF downloader to automate the retrieval of journal articles from online data bases; and PRISMA flow diagrams.

## Installation and Dependencies

`Metagear` has several external dependencies that need to be installed and loaded prior to use in `R`. The first is the `EBImage R` package (Pau et al. 2010) available only from Bioconductor repository.

To properly install `metagear`, use the following script in `R`:

```
# first load Bioconductor resources needed to install the EBImage package.
Uncomment the code below to install by removing the #, run and
accept/download all of its dependencies
#install.packages("BiocManager");
#BiocManager::install("EBImage")
# then load metagear
library(metagear)
```

Finally, for Mac OS users, installation is sometimes not straightforward, as the abstract_screener() requires the Tcl/Tk GUI toolkit to be installed. You can get this toolkit by making sure that the latest X11 application (xQuartz) is installed from here: xquartz.macosforge.org.

## Delegating reference screening efforts to a team

One of the first tasks of a systematic review is to screen the titles and abstracts of study references to assess their relevance for the synthesis project. For example, after a

bibliographic search using Web of Science or PubMed, there may be thousands of references generated; references from experimental studies, modeling studies, review papers, commentaries, etc. These need to be reviewed individually as a first pass to exclude those that do not fit the synthesis project; such as excluding simulation studies that do not report experimental outcomes useful for estimating an effect size.

However, individually screening thousands of references is time-consuming, and large synthesis projects may benefit from delegating this screening effort to a research team. Having multiple people screen references also provides an opportunity to assess the repeatability of these screening decisions.

In this session, we have the following goals:

1. Initialize a dataframe containing bibliographic data (tile, abstract, journal) from multiple study references.
2. Distribute these references randomly to team members.
3. Merge and summarize the screening efforts of this team.

First, let's start by loading and exploring the contents of a sample (smaller sample for practical purposes) of the *new_results* from Part I that contains the bibliographic information of 5691 journal articles. We can use the following code to create *new_results_sample* which consists of 200 studies randomly selected from studies (rows) in the data frame and to subset the columns containing abstracts, journal, titles, volume, page numbers, and authors of these references..

```
library(dplyr)
#randomly select 3.52% of all rows from data frame
set.seed(123)
new_results_sample <- new_results %>% sample_frac(0.0352) %>% select(journal,
abstract, title, volume, pages, author)
dim(new_results_sample)

## [1] 200   6
```

Display the various Journals that these references were published in

```
new_results_sample["journal"]

##
journal
## 1
Journal of dentistry
## 2
Behavior modification
## 3
Behavior therapy
## 4
Zeitschrift fur Psychosomatische Medizin und Psychotherapie
## 5
<NA>
## 6
International forum of allergy & rhinology
## 7
Journal of traumatic stress
## 8
Psychological medicine
## 9
Journal of behavior therapy and experimental psychiatry
```

```
## 10
Annals of the New York Academy of Sciences
```

Our next step is to initialize/prime this dataset for screening tasks. Our goal is to distribute screening efforts to three screeners/reviewers: *Patrick*, *Joseph* and *Barbara*. Here each reviewer will screen a separate subset of these references (a forthcoming example will review how to set up a dual screening design where each member screens the same references). The dataset first needs to be initialized as follows:

```
# prime the study-reference dataset
theRefs <- effort_initialize(new_results_sample)
# display the new columns added by effort_initialize
names(theRefs)
```

```
## [1] "STUDY_ID"  "REVIEWERS" "INCLUDE"   "journal"   "abstract"  "title"
## [7] "volume"    "pages"     "author"
```

Note that the `effort_initialize()` function added three new columns: **STUDY_ID** which is a unique number for each reference (e.g., from 1 to 11), **REVIEWERS** an empty column with NAs that will be later populated with our reviewers (e.g., Patrick, Joseph and Barbara), and finally the **INCLUDE** column, which will later contain the screening efforts by the two reviewers.

Screening efforts are essentially how individual study references get coded for inclusion in the synthesis project; currently the *INCLUDE* column has each reference coded as *not vetted*, indicating that each reference has yet to be screened.

Our next task is to delegate screening efforts to our three reviewers Patrick, Joseph and Barbara. Our goal is to randomly distribute these references to each reviewer.

```
# randomly distribute screening effort to a team
theTeam <- c("Patrick", "Joseph", "Barbara")
set.seed(123)
theRefs_unscreened <- effort_distribute(theRefs, reviewers = theTeam)
# display screening tasks
theRefs_unscreened[c("STUDY_ID", "REVIEWERS")]
```

```
##     STUDY_ID REVIEWERS
## 1          1   Barbara
## 2          2   Barbara
## 3          3   Patrick
## 4          4   Barbara
## 5          5   Barbara
## 6          6   Patrick
## 7          7    Joseph
## 8          8   Patrick
## 9          9   Barbara
## 10        10   Barbara
```

The screening efforts can also be delegated unevenly, such as below where Patrick will take on 80% of the screening effort:

```
# randomly distribute screening effort to a team, but with Patrick handling
80% of the work
set.seed(123)
theRefs_unscreened <- effort_distribute(theRefs, reviewers = theTeam, effort
```

```
= c(80, 10, 10))
theRefs_unscreened[c("STUDY_ID", "REVIEWERS")]

##     STUDY_ID REVIEWERS
## 1          1   Patrick
## 2          2    Joseph
## 3          3   Patrick
## 4          4   Barbara
## 5          5    Joseph
## 6          6   Patrick
## 7          7   Patrick
## 8          8   Patrick
## 9          9   Barbara
## 10        10   Barbara
```

The effort can also be redistributed with the `effort_redistribute()` function. In the above example, we assigned Patrick 80% of the work. Now lets `redistribute half of Patrick's` work to a new team member *Leah*.

```
set.seed(123)
theRefs_Leah <- effort_redistribute(theRefs_unscreened,
                                    reviewer = "Patrick",
                                    remove_effort = 50, # move 50% of Patrick's work to Leah
                                    reviewers = c("Patrick", "Leah")) # team members loosing
and picking up work
theRefs_Leah[c("STUDY_ID", "REVIEWERS")][131:140,] # I am showing rows 131:140 using
[131:140,] for demo as the first ones only show Joseph. Remove [131:140,] for your case.

##     STUDY_ID REVIEWERS
## 124      124   Patrick
## 125      125      Leah
## 126      126   Patrick
## 127      127      Leah
## 128      128   Patrick
## 129      129      Leah
## 130      130   Patrick
## 132      132   Patrick
## 133      133      Leah
## 134      134      Leah
```

The references have now been randomly assigned to Patrick, Joseph and Barbara. The whole initialization of the reference dataset with `effort_initialize()` can be abbreviated with

```
effort_distribute(new_results_sample,  reviewers  =  c("Patrick",  "Joseph",
"Barbara"), initialize = TRUE)
```

Now that screening tasks have been distributed, the next stage is for reviewers to start the manual screening of each assigned reference. This is perhaps best done by providing a separate file of these references to Patrick, Joseph and Barbara. They can then work on screening these references separately and remotely. The `effort_distribute()` function can also save to file each reference subset; these can be given to Patrick, Joseph and Barbara to start their work. This is done by setting the `save_split` parameter to TRUE.

```
# randomly distribute screening effort to a team, but with Patrick handling
80% of the work,
# but also saving these screening tasks to separate files for each team
member
set.seed(123)
```

```
theRefs_unscreened <- effort_distribute(theRefs, reviewers = theTeam, effort
= c(80, 10, 10), save_split = TRUE)

## 3 files saved in: /Users/panj/Documents/ISADS/Workshops and
seminars/Advanced Course

theRefs_unscreened[c("STUDY_ID", "REVIEWERS")]

##     STUDY_ID REVIEWERS
## 1          1   Patrick
## 2          2    Joseph
## 3          3   Patrick
## 4          4   Barbara
## 5          5    Joseph
## 6          6   Patrick
## 7          7   Patrick
## 8          8   Patrick
## 9          9   Barbara
## 10        10   Barbara


list.files(pattern = "effort")

## [1] "effort_Barbara.csv"  "effort_Joseph.csv"
## [2] "effort_Patrick.csv"
```

These two `effort_*.csv` files contain the assigned references for Joseph, Barbara and Patrick. These can be passed on to each team member so that they can begin screening/coding each reference for inclusion in the synthesis project.

## Screening abstracts of references

In this example, I coded each references from Barbara and Joseph as either YES, NO, or MAYBE. These files now contain the completed screening efforts.

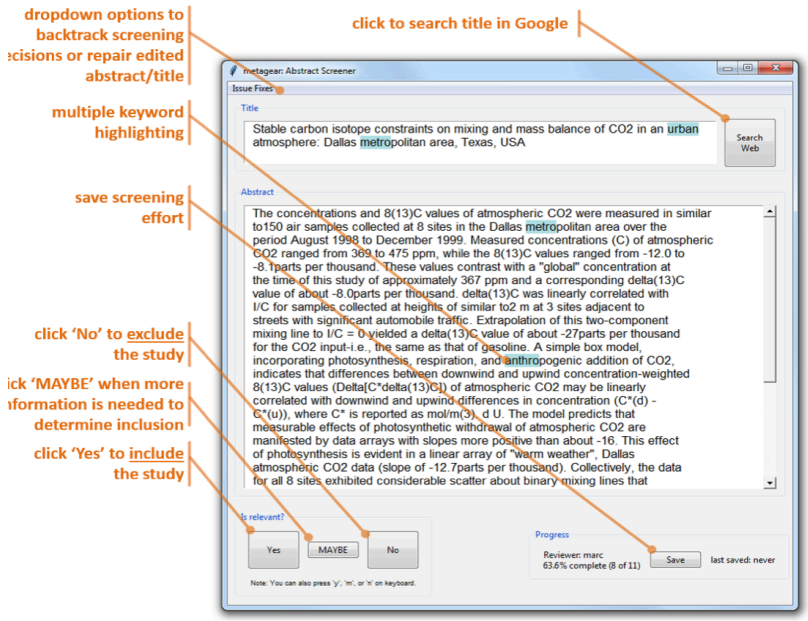
Initialize screener GUI. Run `?abstract_screener` for more details about the options in the function.

```
abstract_screener("effort_Barbara.csv", aReviewer = "Barbara", reviewerColumnName =
"REVIEWERS", unscreenedColumnName = "INCLUDE", abstractColumnName = "abstract",
titleColumnName = "title")

## <Tcl>

abstract_screener("effort_Joseph.csv", aReviewer = "Joseph", reviewerColumnName =
"REVIEWERS", unscreenedColumnName = "INCLUDE", abstractColumnName = "abstract",
titleColumnName = "title")

## <Tcl>
```

The GUI itself will appear as a single window with the first title/abstract listed in the `.csv` file. If abstracts have already been screened/coded, it will begin at the nearest reference labeled as *not vetted*. The SEARCH WEB button opens the default browser and searches Google with the title of the reference. The *YES*, *MAYBE*, *NO* buttons, which also have shortcuts ALT-Y and ALT-N, are used to code the inclusion/exclusion of the reference. Once clicked/coded the next reference is loaded. The SAVE button is used to save the coding progress of screening tasks. It will save coding progress directly to the loaded `.csv` file.

Closing the GUI and not saving will result in the loss of screening efforts relative to last save.

Here's what to expect with this GUI (note that depending on the platform running R, the layout of this GUI will differ slightly):




*Metagear Abstract Screener GUI*

## Merging screened references

Once the screening is complete, we can then merge these files into a complete dataset. References were been as *YES* or *NO* for inclusion, but can also be coded as *MAYBE* if bibliographic information is missing or there is inadequate information to make a proper assessment of the study. The `abstract_screener()` function was used to facilitate this screening process in the example above where I randomly coded each references allocated to Barbara and Joseph as either *YES*, *NO*, or *MAYBE*. These files now contain the completed screening efforts.

We can merge these two files with the completed screening efforts using the `effort_merge()` function, as well as summarize the outcome of screening tasks using the `effort_summary()` function.

```
# merge the effort_Barbara.csv and effort_Joseph.csv
screenedTeam <- c("Barbara", "Joseph")
theRefs_screened <- effort_merge(reviewers = screenedTeam)
theRefs_screened[c("STUDY_ID", "REVIEWERS", "INCLUDE")]

theSummary <- effort_summary(theRefs_screened)
```

| STUDY_ID <int> | REVIEWERS <chr> | INCLUDE <chr> |
|---|---|---|
| 4 | Barbara | YES |
| 9 | Barbara | maybe |
| 10 | Barbara | NO |
| 14 | Barbara | maybe |
| 15 | Barbara | NO |
| 22 | Barbara | YES |
| 23 | Barbara | YES |
| 27 | Barbara | maybe |
| 45 | Barbara | NO |
| 50 | Barbara | NO |

```
## === SCREENING EFFORT SUMMARY ===
##
   80 candidate studies identified
   9 studies excluded
   11 challenging studies needing additional screening
  ----
   20 TOTAL SCREENED

=== SCREENING DESIGN SUMMARY ===
##
## === SCREENING DESIGN SUMMARY ===
##
```

| | maybe <dbl> | NO <dbl> | not vetted <dbl> | YES <dbl> | TOTAL <dbl> | % <dbl> |
|---|---|---|---|---|---|---|
| Barbara | 6 | 5 | 40 | 9 | 60 | 50 |
| Joseph | 5 | 4 | 40 | 11 | 60 | 50 |
| TOTAL | 11 | 9 | 80 | 20 | 120 | 100 |

3 rows

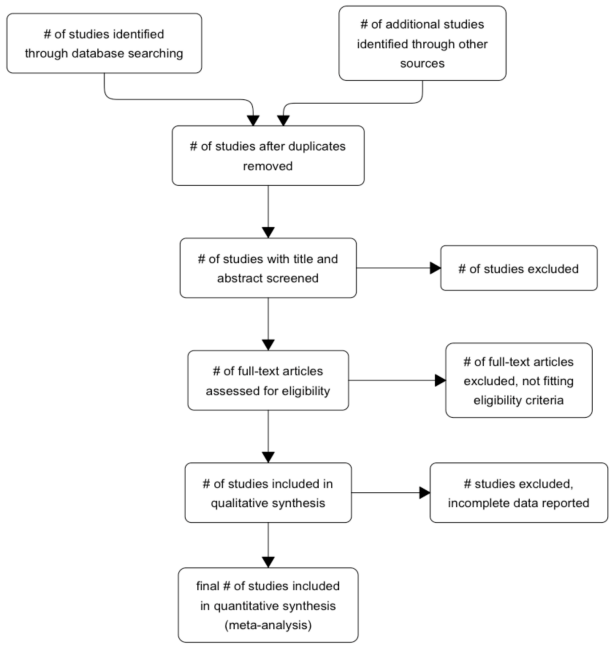
The summary of screening tasks describes the outcomes of which references had studies appropriate for the synthesis project, while also outlining which need to be re-assessed. The team should discuss these challenging references and decide if they are appropriate for inclusion or track down any additional/missing information needed to make proper assessment of their inclusion.

## Part III Generating PRISMA plots

PRISMA plots (preferred reporting items for systematic reviews and meta-analyses), or PRISMA flow diagrams, are an important and simple way to present the flow of information on how studies were found, collated, and screened for systematic reviews and meta-analysis (Liberati et al. 2009). Generally, they depict from top to bottom the original number of studies identified through bibliographic databases (and other sources) and how this population of studies was culled for inclusion into the synthesis project.

`Metagear` offers an easy way to generate PRISMA plots, this requires a list of the *phases* of the screening process. It also requires certain phases to be labeled to properly depict the start (with the string `START_PHASE:`) and exclusion (string `EXCLUDE_PHASE:`) phases of the flow diagram. Below is a quick example.

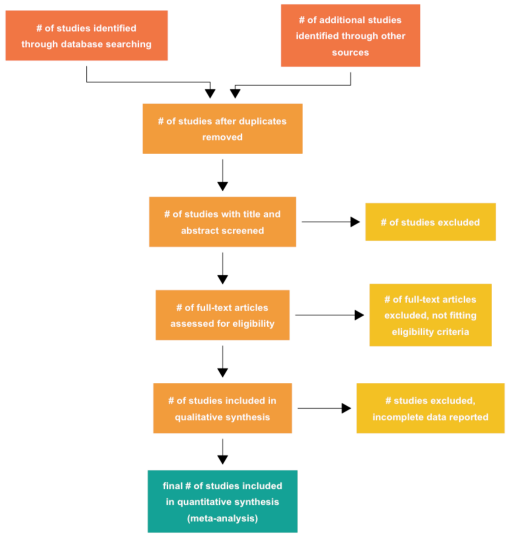
```r
# load package
library(metagear)

phases <- c("START_PHASE: # of studies identified through database searching",
            "START_PHASE: # of additional studies identified through other sources",
            "# of studies after duplicates removed",
            "# of studies with title and abstract screened",
            "EXCLUDE_PHASE: # of studies excluded",
            "# of full-text articles assessed for eligibility",
            "EXCLUDE_PHASE: # of full-text articles excluded, not fitting eligibility
criteria",
            "# of studies included in qualitative synthesis",
            "EXCLUDE_PHASE: # studies excluded, incomplete data reported",
            "final # of studies included in quantitative synthesis (meta-analysis)")
thePlot <- plot_PRISMA(phases, excludeDistance = 1, colWidth = 50)
```



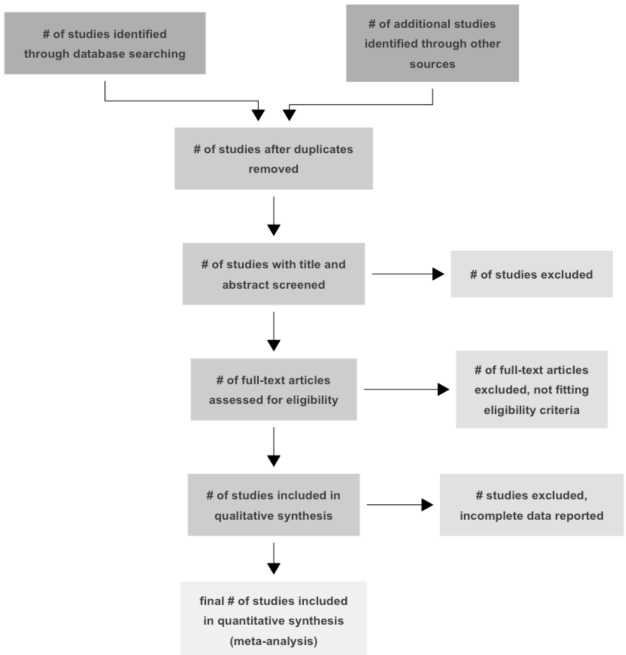

## Generating different PRISMA plot layouts

There are also several different PRISMA plot layouts available to generate simpler or more colourful schemes. Below are a few of the various designs available, and for example, here the `cinnamonMint` scheme can be generated using:

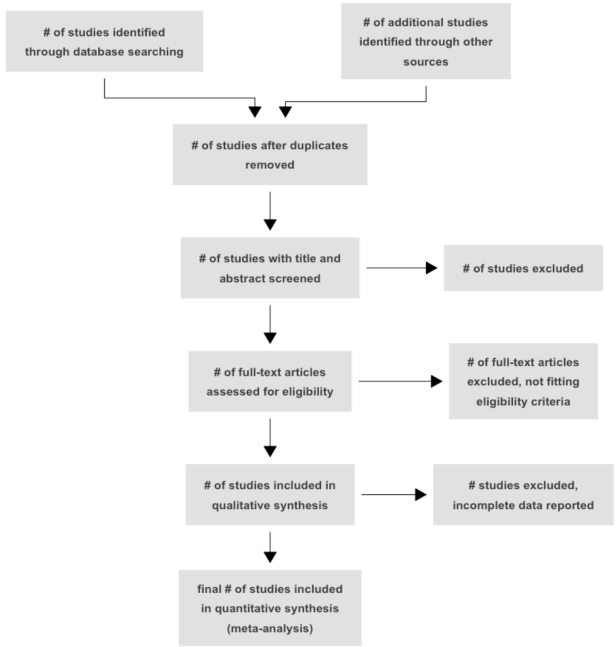
```r
plot_PRISMA(phases, design = "cinnamonMint")
```



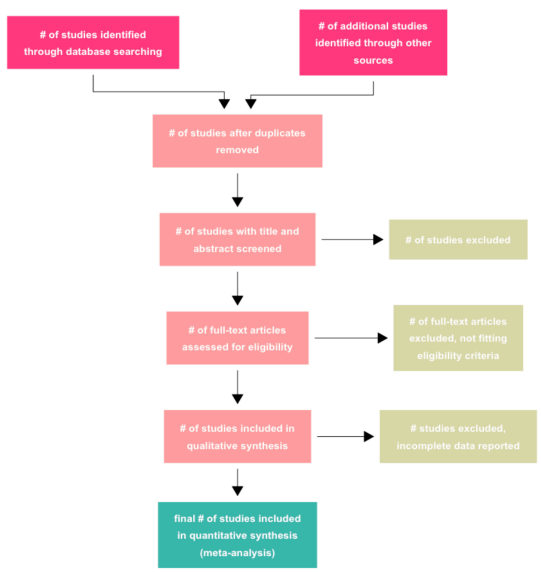

```
plot_PRISMA(phases, design = "grey")
```




```
plot_PRISMA(phases, design = "greyMono")
```




```
plot_PRISMA(phases, design = "vintage")
```




It is also possible to individually customize various design features of the PRISMA plot. Here are some of the features that can be modified:

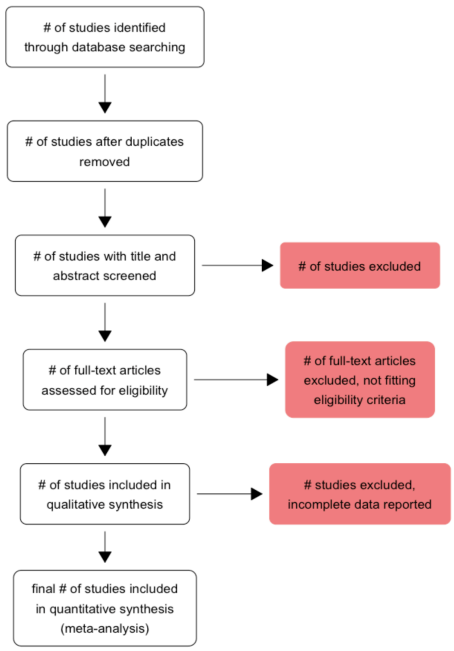
Parameter options

- `S`- color of start phases (default: white)
- `P`- color of the main phases (default: white)
- `E`- color of the exclusion phases (default: white)
- `F`- color of the final phase (default: white)
- `fontSize`- the size of the font (default: 12)
- `fontColor`- the font color (default: black)
- `fontFace`- either plain, bold, italic, or bold.italic (default: plain)

- `flatArrow`- arrows curved when FALSE (default); arrows square when TRUE
- `flatBox`- boxes curved when FALSE (default); boxes square when TRUE

Here's a quick example on how to change the color of the exclusion phases:

```
phases <- c("START_PHASE: # of studies identified through database searching",
            "# of studies after duplicates removed",
            "# of studies with title and abstract screened",
            "EXCLUDE_PHASE: # of studies excluded",
            "# of full-text articles assessed for eligibility",
            "EXCLUDE_PHASE: # of full-text articles excluded, not fitting eligibility
criteria",
            "# of studies included in qualitative synthesis",
            "EXCLUDE_PHASE: # studies excluded, incomplete data reported",
            "final # of studies included in quantitative synthesis (meta-analysis)")

# PRISMA plot with custom layout
thePlot <- plot_PRISMA(phases, design = c(E = "lightcoral", flatArrow = TRUE))
```




## Notes on PRISMA plotting since metagear v. 0.1 and 0.2

Previous versions of `metagear` (v. 0.2 and 0.1) offered a more flexible version of `plot_PRISMA()` that allowed for proper rescaling of PRISMA objects when the user manually changed the window size of the plot. Unfortunately, this version did not load well when bundled with the package and yielded unusual plots. These versions are archived and allow for higher-quality PRISMA plots; these old functions can be downloaded here and work best when not loaded with `metagear`.

# References

Lajeunesse, M.J. (2016) Facilitating systematic reviews, data extraction and meta-analysis with the metagear package for R. Methods in Ecology and Evolution 7: 323-330.article link

Pau, G., Fuchs, F., Sklyar, O., Boutros, M. and Huber, W. (2010). EBImageâ€"an R package for image processing with applications to cellular phenotypes. Bioinformatics 26: 979-981.

Liberati, A., Altman, D.G., Tetzlaff, J., Mulrow, C., Gotzsche, P.C., Ioannidis, J.P., Clarke, M., Devereaux, P.J., Kleijnen, J., and Moher, D. (2009) The PRISMA statement for reporting systematic reviews and meta-analyses of studies that evaluate health care interventions: explanation and elaboration. PLoS Medicine 6: e1000100.